Exp-1 BRESENHAM'S LINE DRAWING ALGORITHM (|m| < 1)

Aim-

To implement Bresenham's line drawing Algorithm (|m| < 1) in python for drawing a line when the slope.

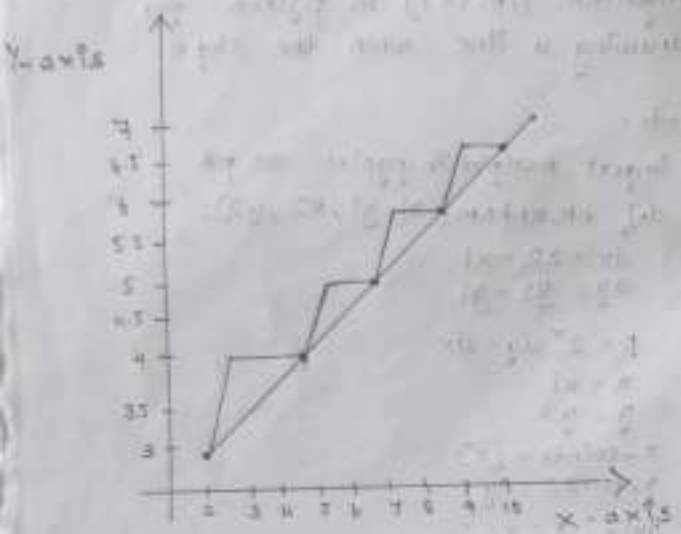
Code-

```
import matplotlib.pyplot as plt
def bresenham( x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    p = 2 * dy - dx
    x = x1
    y = y1
x_points = [x]
y_points = [y]
while x < x2:
    x += 1
    if p < 0:
        p = p + 2 * dy
    else:
        y += 1
        p = p + 2 * dy - 2 * dx
    x_points.append (x)
    y_points.append (y)
plt.plot (x_points, y_points, 'bo-')
plt.title ("Bresenham Line Drawing")
plt.xlabel ("X-axis")
plt.ylabel ("Y-axis")
```

Input

(2, 3)

(10, 7)



Output-

(2,3)

(3,4)

(4,4)

(5,5)

(6,5)

(7,6)

(8,6)

(9,7)

(10,7)

o/p verified

plt.grid(True)

plt.show()

Example

x1, y1 = 2, 3

x2, y2 = 10, 7

bresenham(x1, y1, x2, y2)

RESULT-

The program was executed successfully. Bresenham's line drawing Algorithm for the input whose slope |m| < 1.

28/7/26 Exp-2 BRESENHAM'S LINE, CIRCLE, ELLIPSE, DRAWING ALGORITHM.

Aim -

To Implement Bresenham's line, circle, ellipse drawing algorithm in python

Program

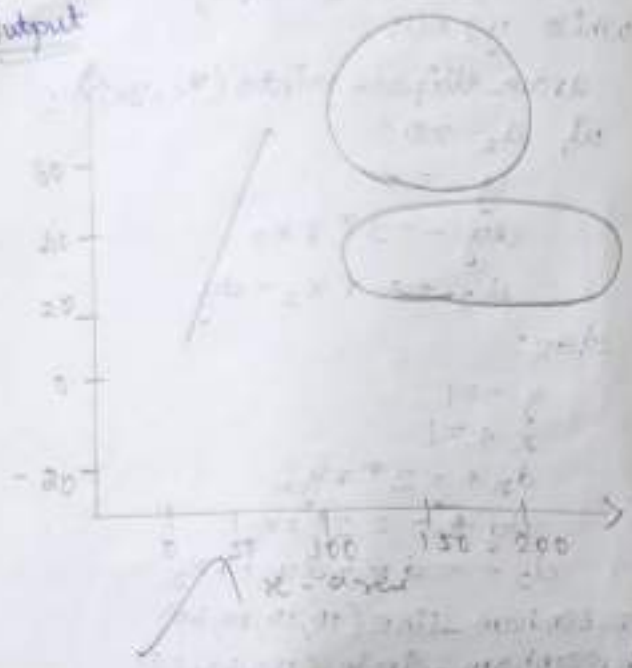
```
import matplotlib.pyplot as plt
line_points = [ ]
def plot_line (x, y):
    line_points.append ((x, y))
def breshmam_line (x1, y1, x2, y2)
    dx = abs(x2 - x1)
    dy = abs (y2 - y1)
  sx = 1 if x2 > x1 else -1
  sy = 1 if y2 > y1 else -1
  err = dx - dy
while True :
  plot_line (x1, y1)
if x1 == x2 and y1 == y2 :
    break
  e2 = 2 * err
if e2 > -dy :
    err -= dy
    x1 += sx
if e2 < dx :
    err += dx
    y1 += sy.
```

```
Circle_points=[]
def plot_circle (x,y):
    circle_points.append ((x,y))
def draw_circle_points (xc, yc, x, y):
    points = [(xc+x, yc+y),(xc-x,yc+y),
        (xc+x, yc-y),(xc-x, yc-y),
        (xc+y, yc+x),(xc-y, yc+x),
        (xc+y, yc-x),(xc-y, yc-x)]
for p in points:
    plot_circle (*p)
def bresenham_circle(xc, yc):
    x=0
    y=r
    d=3-2*r
while x<=y:
    draw_circle_points( xc, yc, x, y)
    if d<0:
        d+ = 4*x+6
    else:
        d+ = 4*(x-y)+10
        y-=1
        x+=1
Ellipse_points=[]
def plot_ellipse (x,y):
    ellipse_points.append((x,y))
def draw_points.append (xc, yc, x, y):
```

```
points = [(xc+x, yc+y),
          (xc-x, yc+y),
          (xc+x, yc-y),
          (xc-x)]
for p in plots:
    plot_ellipse(*p)
def bresenham_ellipse(xc, yc, rx, ry):
    x = 0
    y = ry
    rx2 = rx * rx
    ry2 = ry * ry
    dx = 2 * ry2 * x
    dy = 2 * rx2 * y
    d1 = ry2 - rx2 * ry + 0.25 * rx2
    while dx < dy:
        draw_ellipse_points(xc, yc, x, y)
        if d1 < 0:
            x += 1
            dx += 2 * ry^2
            d1 += dx + ry^2
        else:
            x += 1
            y -= 1
            dx += 2 * ry^2
            dy -= 2 * rx^2
            d1 += dx - dy + ry2
```

```
d2 = (ry2 * (x+0.5) ** 2) + (rx2 * (y-1)
    ** 2) - (rx2 * ry2)
while y > 0:
    draw_ellipse_points (xc, yc, x, y)
    if d2 > 0:
        y -= 1
        dy -= 2 * rx2
        d2 += rx2 - dy
    else:
        y -= 1
        x += 1
        dx += 2 * ry2
        dy -= 2 * rx2
        d2 += dx - dy + rx2
bresenham_line(10,10,80,60)
bresenham_circle(180,60,25)
bresenham_ellipse(220,60,40,20)
plt.figure(figsize=(10,5))
plt.plot([p[0] for p in line_points]
plt.plot([p[0] for p in ellipse_points],
         p[1] for p in ellipse_points],
         marker='.',
         linestyle='None',
         markersize=2,
         color='black',
         label='Ellipse')
```

Output



plt.title ("Bresenhman Line")
plt.xlabel (x-axis)
plt.ylabel (y-axis)
plt.axis ("equal")
plt.grid (True)

RESULT -

The program was executed succesfully

Ex: 3 Digital differential analyzer (DDA) line drawing algorithm

Aim:

To Create a square do using DDA line drawing algorithm.

code:

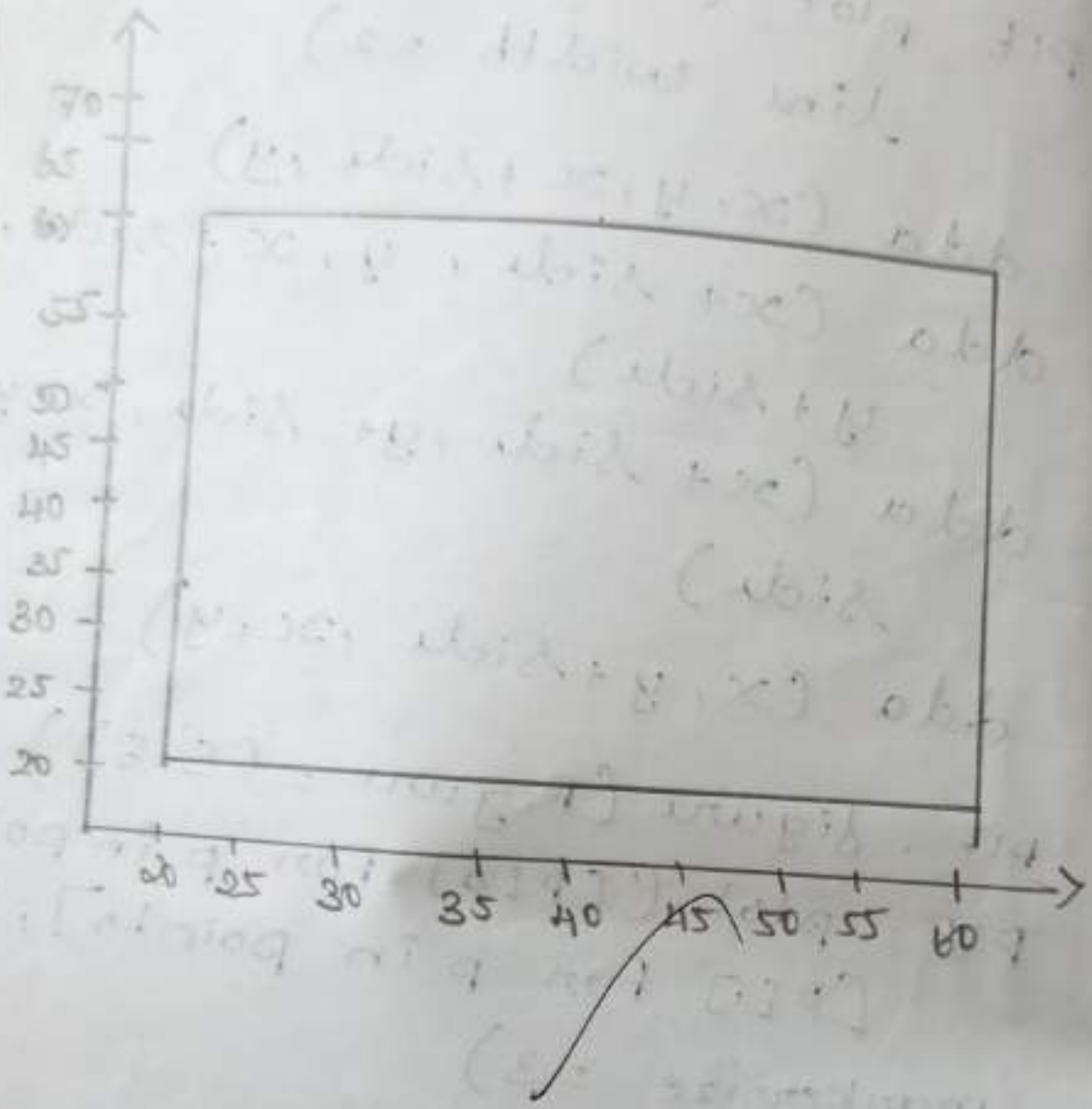
```
import matplotlib.pyplot as plt
points = []
def dda (x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    steps = max (abs (dx), abs
                 (dy))
    x_inc = dy / steps
    y_inc = dy / steps

    x = x1
    y = y1      x = points = []
                y = points []

for i in range (steps + 1):
    points.append ((round (x).
        round (y))
    x + = x_inc
    y + y_inc
```

```
x = 20
y = 20
side = 40
plt.plot (x_points, y_points, 'k',
        line width = 2)
dda (x, y, x + side, y)
dda (x + side, y, x + side,
    y + side)
dda (x + side, y + side, x, y +
    side)
dda (x, y + side, x, y)
plt.figure (Figure = (5, 5))
plt.plot ([p[0] for p in points],
    [p[1] for p in points], 'k',
    markersize = 3)
plt.title ("Square using DDA algorithm")
plt.xlabel ("x-axis")
plt.ylabel ("y-axis")
plt.grid (True)
plt.gca().set_aspect ('equal')
plt.show()
```

OUTPUT -



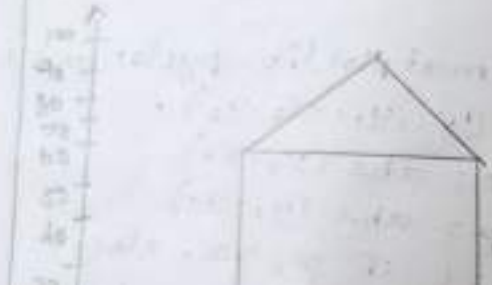
EXP: 4 BUILD AN OBJECT USING BRSENHAM ALGORITHM.

AIM: To Build an object using bresenham algorithm.

Code:

```
import matplotlib.pyplot as plt
def line (x1, y1, x2, y2):
    dx = abs (x2 - x1)
    dy = abs (y2 - y1)
    sx = 1 if x2 > x1 else -1
    sy = 1 if y2 > y1 else -1
    err = dx - dy
    x = []
    y = []
while True:
        x.append (x1)
        y.append (y1)
        if x1 == x2 and
y1 == y2:
        break:
        e2 = 2 * err
        if e2 > -dy:
            err - = dy
            x1 + = sx
```

Output:



plt.plot (x, y, 'k')
line (20, 20, 20, 60)
line (20, 60, 60, 60)
line (60, 60, 60, 20)
line (60, 20, 20, 20)
line (30, 60, 40, 80)
line (40, 80, 60, 60)
line (35, 20, 25, 40)
line (45, 20, 55, 40)
line (35, 40, 25, 40)

using bresenham

EX:P 5

HOUSE DRAWING USING THE BRESENHAM'S ALGORITHM

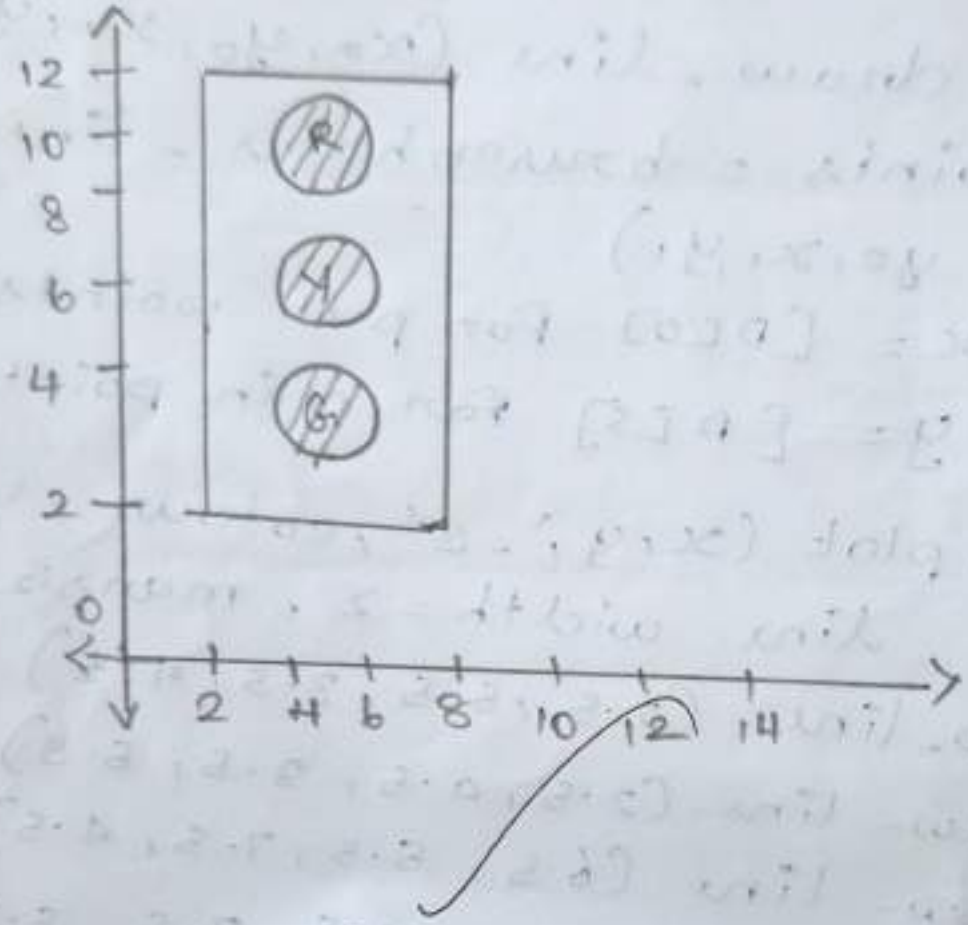
Aim: To Implement brusenham's house drawing algorithm in python with the given co-ordi -nates

program:

```
import matplotlib.pyplot as plt
def brusenham-line (x0, y0, x1, y1);
    dx = abs (x1 - x0)
    dy = abs (y1 - y0)
    Sx = 1 if x0 < x1 else -1
    Sy = 1 if y0 < y2 else -1
    err = dx - dy
    points = []
while true:
    points.append ((x0, y0))
    if x0 == x1 and y0 == y1:
        break
    e2 = 2 * err
    if e2 > -dy:
        err -= dy
```

```
        x0 += sx
    if e2 > dx
        err += dx
        y0 += sy
    return points
def draw_line(x0, y0, x1, y1)
    points = bresenham_line(x0,
        y0, x1, y1)
    x = [p[0] for p in points]
    y = [p[1] for p in points]
plt.plot(x, y, '-s', colour = 'purple',
        line width = 2, marsize = 5)
draw_line(2.5, 5.5, 3.5, 4.5)
draw_line(2.5, 4.5, 3.5, 5.5)
draw_line(6.5, 5.5, 7.5, 4.5)
draw_line(6.5, 4.5, 7.5, 5.5)
plt.title("House with windows
        and fence using bresenham's
line algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.gca().set_aspect('equal')
plt.xticks(range(0,17))
```

OUTPUT



Exp-6

RECTANGLE USING BRESENHAM'S ALGORITHMS

Given:

Width = 8, height = 5, vertex (2, 2)

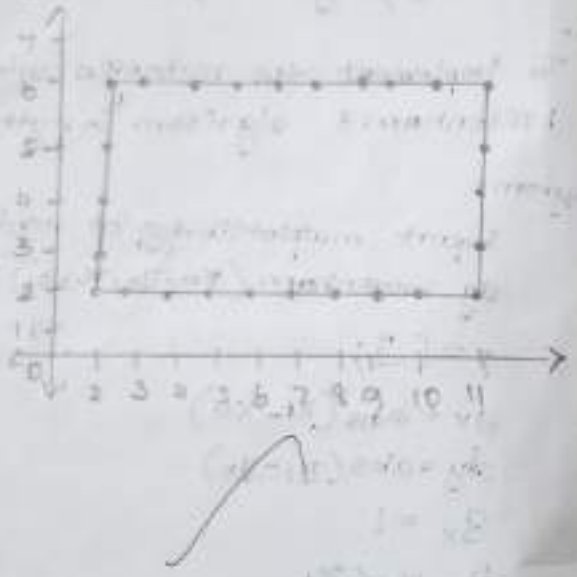
Aim-

To implement the rectangle using the bresenham's algorithm in python.

Program-

```
import matplotlib.pyplot as plt
dy bresenham (x0, y0, x, y1):
p = [];
dx = abs (x1 - x0)
dy = abs (y1 - y0)
Sx = 1
if x0 < x1
  else -1; Sy - 1 if y0 < y1 else -1
e = dx - dy
while 1:
  p.append ((x0, y0))
if (x0, y0) == (x1, y1): return p
e2 = 2*e
if e2 >= dy : e -= dy; x0 += Sx
```

OUTPUT -



```
if p < 0 : dx = dx ; dy += 8y
x, y, w, h = 2, 2, 8, 5
xe, ye = x+w-1, y+h-1
for a, b, c, d in [(x, y, xe, y),
    (x, ye, xe, ye), (x, y, x, ye), (xe, y, xe, ye)]:
    p = bresenham (a, b, c, d)
    plt.plot (*zip(*p), 'bo-')
    plt. grid ()
    plt. axis ('equal')
    plt. show ()
```

RESULT -

The rectangle is drawn successfully using bresenham's algorithm.

Exp no-7

TRAFFIC SIGNAL USING BRESENHAM'S ALGORITHM

Aim:-

To implement the program to draw the traffic signal using bresenham's Algorithm.

PROGRAM-

```
import matplotlib.pyplot as plt
from matplotlib.patches import
                    circle, rectangle
fig, ax = plt.subplots()
box = Rectangle((2,1), 4, 10), facecolor=
                    "black")
ax.add_patch(box)
red = Circle((4,8), 1, facecolor="red",
        edgecolor = "white")
ax.add_patch(red)
yellow = circle((4, 6), 1, facecolor=
                "yellow", edgecolor="white")
ax.add_patch(yellow)
green = circle((4,4), 1, facecolor="green",
        edgecolor= "white")
ax.add_patch(green)
```

```
ax.add_patch(green)
ax.set_xlim(0,8)
ax.set_ylim(0,12)
ax.set_aspect("equal")
ax.grid(True)
plt.title("Traffic Signal")
plt.show()
```

OUTPUT



RESULT

The Program for the traffic signal using Bresenhams Algorithm using the bresenham's algorithm.

Exp-8

SMILING FACE WITH EYES, NOSE, MOUTH USING BRESENHAM'S ALGORITHM

AIM -

To Implement the Program to draw a Smiling face with eyes, nose, mouth using bresenham's Algorithm.

PROGRAM

```
import matplotlib.pyplot as plt
import mat
def c(x,y,r,col);
p = [(x+r*math.cos(math.radians(a))
y+r*math.sin(math.radians(a))
for a in range(361)]
x, y = zip(*p); plt.fill(x,y,col);
plt.plot(x,y,'k') c((5,3,3);
'yellow')
for x in [3,8,6,2]:
c(x,6,2,.55,'white')
c(x,6.2,.25,'black')
plt.plot([5,4.7,5.2],[5.8,5.1,5.1],
'k', lw=2)
```

```
x = [3.8 + i*0.085 for i in orange(41)]
t = [4.1 + .25 * math.sin(math.pi*
  i/40) for i in range(41)] plt.fill(
x, y, 'red'); plt.plot(x, y, 'k', lw=2)
  x = [4 + i*, 1 for i in range(20)]
plt.plot(x, y, 'pink', lw = 4)
plt.axis('equal')
plt.xlim(1,9); plt.ylim(1,9)
plt.grid()
plt.title("Smiling face")
plt.show()
```

RESULT –

The program to implement the Smiling face has been done successfully.

Exp no-9

IMPLEMENTION SCAN LINE CONVERSION OF POLYGION & USE FLOOD FILL ALGORITHM.

Aim-

To implement scan line conversion of polygon & use flood fill algorithm

Program-

```
import matplot pyplot as plt
from matplotlib patches import
                        Polyon, Rectangle
polygon = [(2,2) (8,2) (7,6) (5,8)
                        (2,6)]
colours = ['red', 'orange', 'yellow', 'green',
          'blue', 'purple', 'pink']
fig, ax = plt.subplot()
boundary = Polygon( polygon, closed=True,
fill=False, edgecolor = 'black', linewidth=2)
ax.add_patch (boundary)
ymin = min (y for x, y in polygon)
ymax = max ( y for x, y in polygon)
for y in range (ymin, ymax):
  intersection = [ ]
for i in range ( len (polygon)):
  x1, y1 = polygon [i]
```

```
x2, y2 = polygon[(i+1) % len(polygon)]
if y1!=y2 and min(y1,y2) <= y < max(y1,y2):
    x = x1 + (y-y1)*(x2-x1)/(y2-y1)
    intersections.append(x)
intersections.sort()
color = colors[(y-ymin) % len(colors)]
for i in range(0, len(intersections), 2):
    if i+1 < len(intersection):
        x_left = intersections[i]
        x_right = intersection[i+1]
        rect = Rectangle((x_left, y),
        (x_right - x_left, 1, facecolor=
        color, edgecolor=None)
        rect.set_clip_path(boundary)
        ax.add_patch(boundary)
        ax.add_patch(rect)
x = [p[0] for p in polygon] + [polygon[0][0]]
y = [p[1] for p in polygon] + [polygon[0][1]]
ax.plot(x, y, 'black', linewidth=2)
```



```
ax.set_xlim(0, 10)
ax.set_ylim(0, 10)
ax.set_xticks(range(11))
ax.set_aspect("equal")
ax.grid(True)
plt.title("Scan line Conversion-Polygon
                                filling")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Result -

The implementation of Line Conversion of Polygon and use of flood fill is completed Successfully.

Expno-10.

DRAW A BASIC SHAPE & APPLY TRANSFORMATION LIKE ROTATION, TRANSLATION & SCALING.

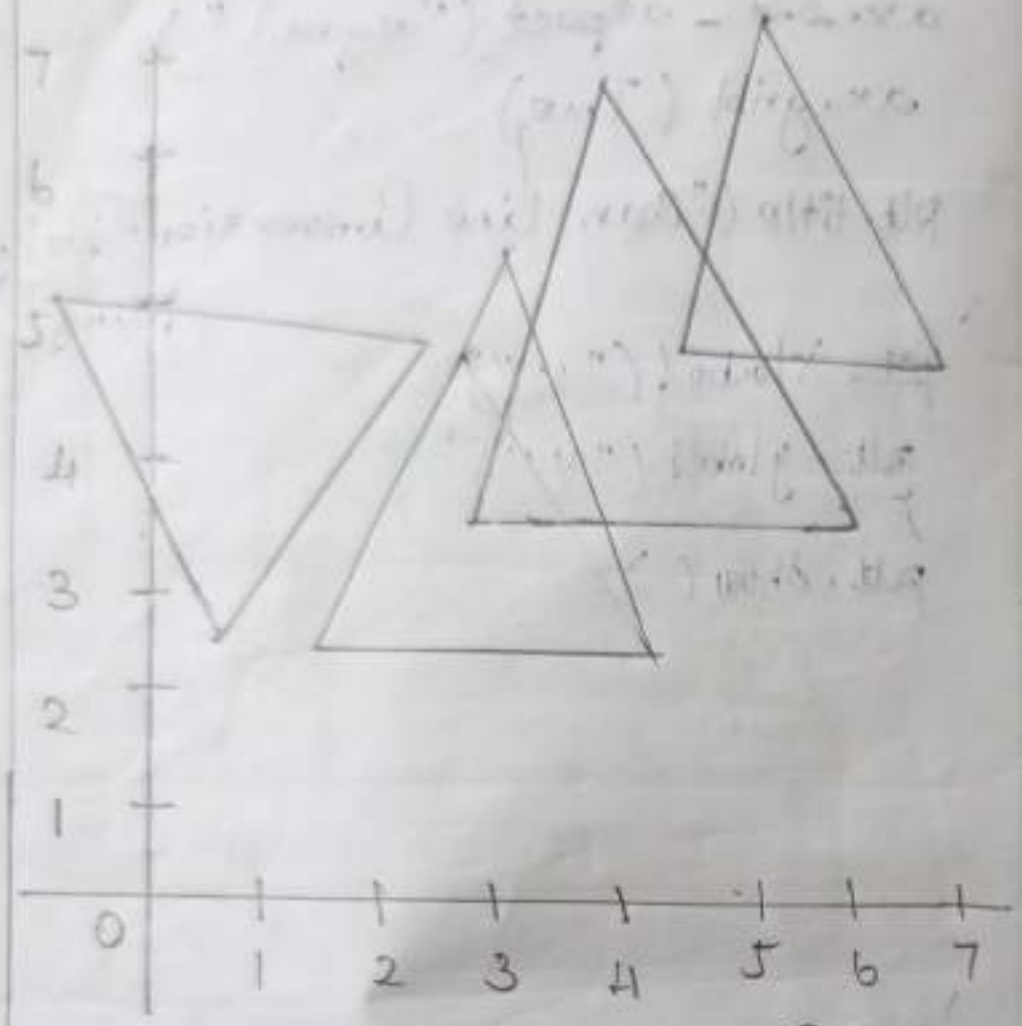
Aim-

To implement a transformation to a basic shape.

Code-

```
import matplotlib.pyplot as plt
import numpy as np
p = np.array([[2,2],[5,2],[3,5],[2,2]])
t = p + [4,2]
s = p * 1.5
a = np.radians(45)
R = np.array([[np.cos(a), -np.sin(a)],
              [np.sin(a), np.cos(a)]])
r = p @ R.T
plt.plot(p[:,0], p[:,1], label="original")
plt.plot(t[:,0], t[:,1], label="Translation")
plt.plot(s[:,0], s[:,1], label="Scaling")
plt.plot(r[:,0], r[:,1], label="Rotation")
plt.axline(0)
plt.axvline(0)
plt.grid()
plt.legend()
plt.show()
```

Result- Hence it is verified successfully.


7
6
5
4
3
2
1
0
1
2
3
4
5
6
7

Exp no-11

DRAW A BASIC RECTANGLE & APPLY TRANSFORMATION.

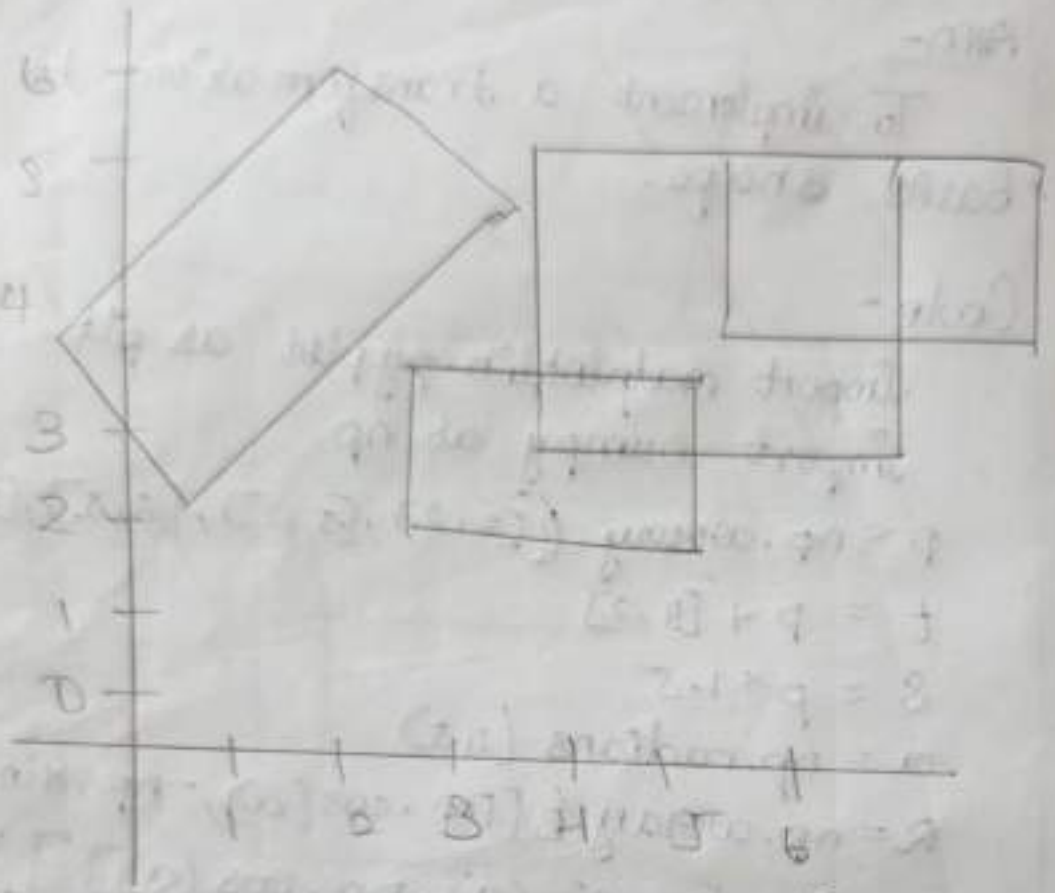
Aim -

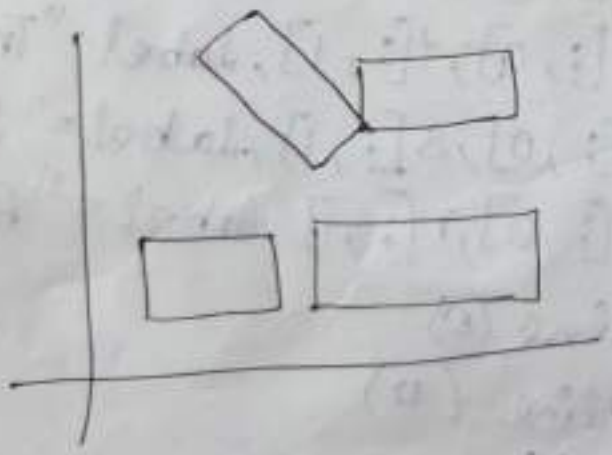
```
import matplotlib.pyplot as plt
import numpy as np
p = np.array([[2,2],[6,2],[6,4],[2,4],
              [2,2]])

t = p + [4,2]
s = p * 1.5
a = np.radians(45)
R = np.array([[np.cos(a), -np.sin
(a)],
np.sin(a), np.cos(a)]])
r = p @ R.T
plt.plot(p[:,0], p[:,1], label="Org")
plt.plot(t[:,0], t[:,1], label="Translation")
plt.plot(s[:,0], s[:,1], label="Scaling")
plt.plot(r[:,0], r[:,1], label="rotation")
plt.axhline(0)
plt.axvline(0)
plt.grid()
plt.legend()
plt.show()
```

Result -

The rectangle is transformed successfully.


6
5
4
3
2
1
0



Ex no: 13

POLYGON CLIPPING USING SUTHERLAND HODGMAN.

Program -

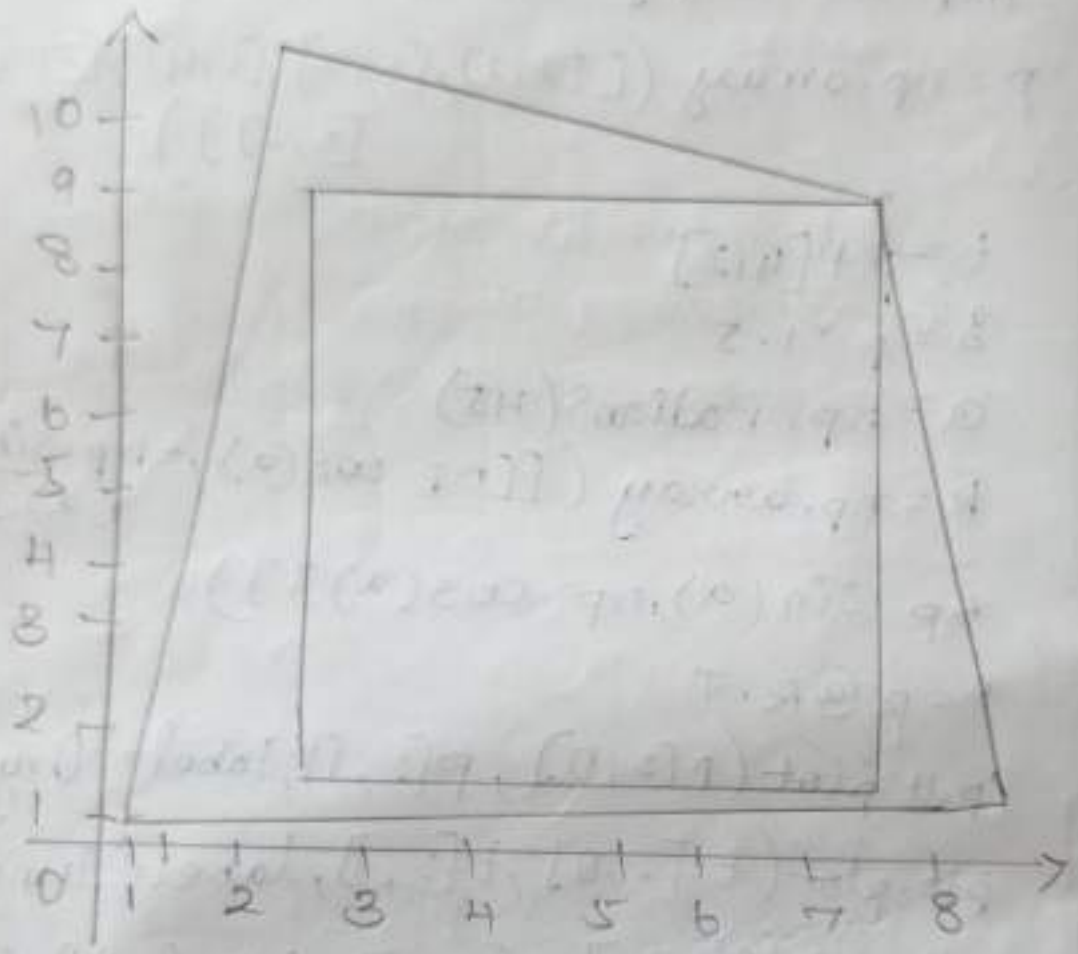
```
import matplotlib.pyplot as plt
p=[(1,1),(7,2),(6,6),(2,7)]
def clip(p,xmin,ymin,ymax):
    return [(max(xmin,min(xmax,x)),
        max(ymin,min(ymax,y))) for
            x,y in p]
c=clip(p,2,6,2,6)
x,y=zip(*(p+[p[0]]))
plt.plot(x,y,label="Original")
x,y=zip(*(c+[c[0]]))
plt.plot(x,y,label="clipped")
plt.grid()
plt.legend()
plt.show()
```

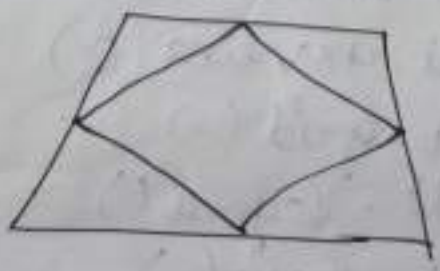
29/9/22

Result -

A Polygon clipping using Sutherland Hodgman.

Output





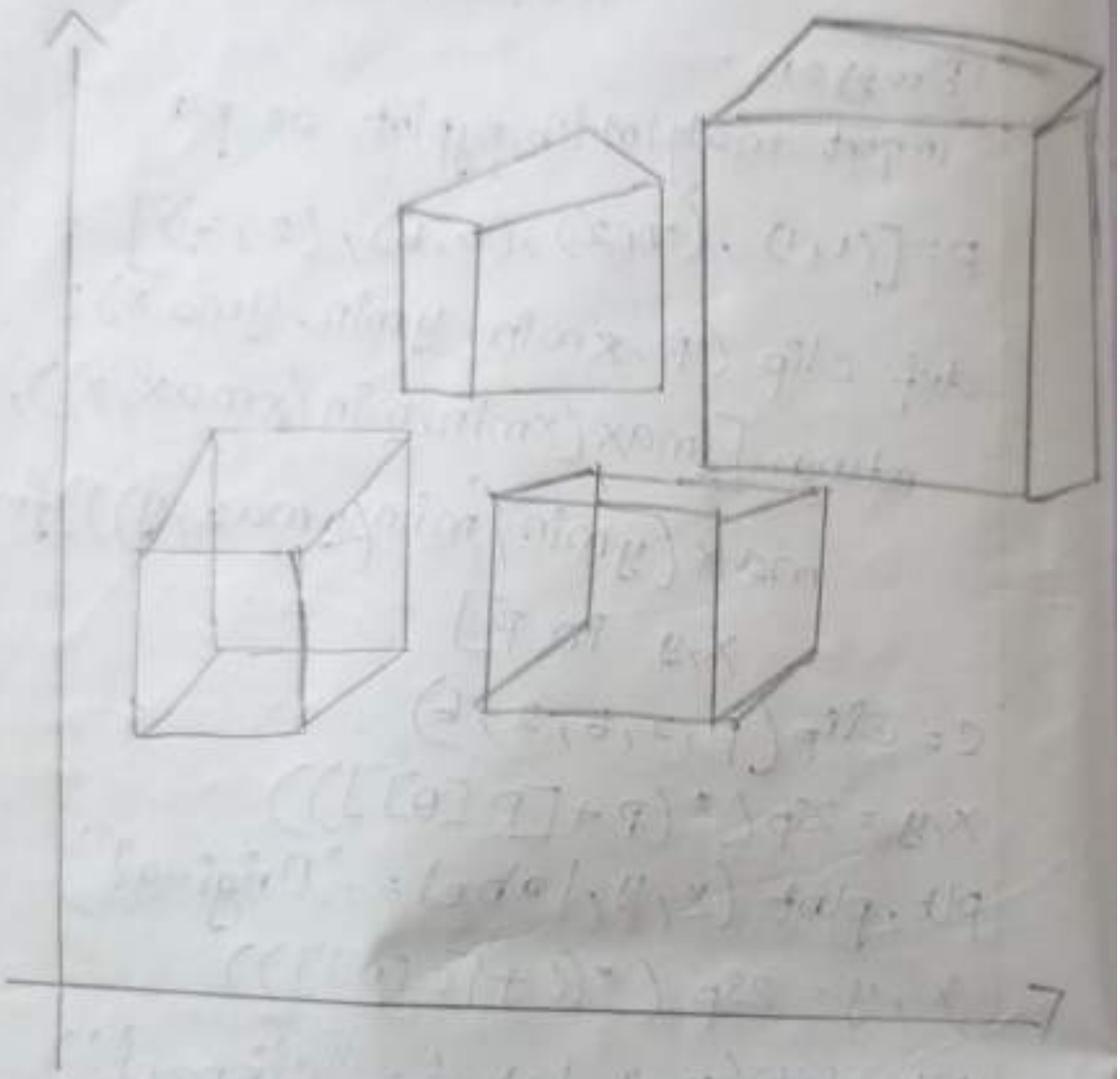
Ex no - 14

3D Transformation on Basic Objects

Aim -

To write a program that allows the user to perform 3D transformations on basic 3D objects like a cube.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import
                                Line3DCollection.
def draw_edges (ax, vertices, edges, color='b'):
    lines=[(vertices[start], vertices[end] for
start, end in edges]
ax.add_collection3d (Line3DCollection)
def translation_matrix (tx, ty, tz):
    return np.array ([[1, 0, 0, tx],
[0, 0, 0, 1]    [0, 1, 0, ty], [0, 0, 1, tz]])
def rotation_mat (sx, sy, sz):
    return np.array [[sx, 0, 0, 0],
        [0, sy, 0, 0], [0, 0, sz, 0], [0, 0, 0,
def apply_transform (vertices, matrix):
    transformed = [ ]
for v in vertices:
    vec = np.array([*v, 1])
```



vertices = [(0,0,0) (1,0,0) (1,1,0) (0,1,0)
(0,0,1) (1,0,1) (1,1,1)]

edges = [(0,1), (1,2) (2,3) (3,0)
(4,5) (5,6) (6,7) (7,4) (0,4)
(1,5) (2,6) (3,7)]

T = translation_mat (2,2,0)
S = Scaling_matrix (1.5, 1.5, 1.5)
R = rotation_mat_z (45)

Result -

The 3D transformation is done successfully

Ex no - 15

RENDERING 3D OBJECTS USING POLYGONS AND SHADING TECHNIQUES

Aim-

To create and render 3D objects using polygonal modelling and apply flat shading & Basic color techniques

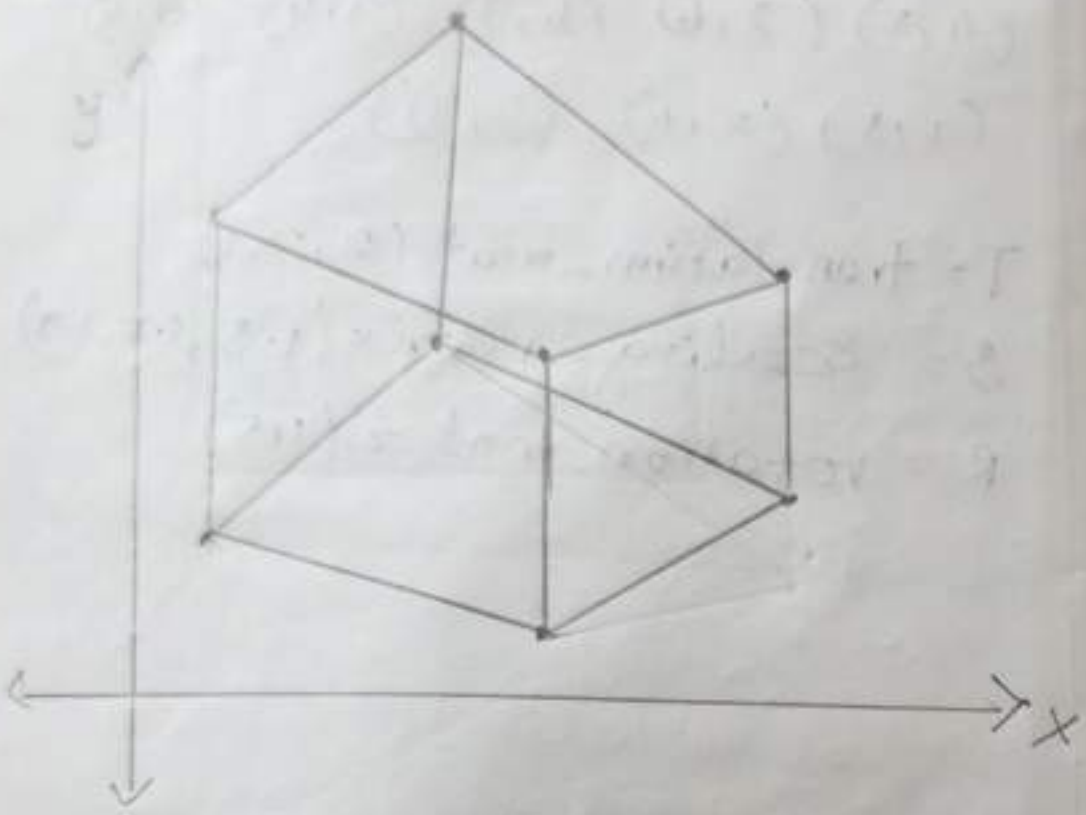
Program-

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import
                        Poly3DCollection
import numpy as np
vertices = np.array([[0,0,0], [1,0,0], [1,1,0],
        [0,1,1], [0,1,0], [0,0,1], [1,0,1] [1,1,1]])
  faces = [[vertices[j] for j in [0,1,2,3]],
     [vertices[j] for j in [4,5,6,7]],
     [vertices[j] for j in [0,1,5,4]]]
colors = ['red', 'blue', 'green', 'yellow', 'orange', 'purple']
  fig = plt.figure()
  ax = fig.add_subplot(111, projection='3d')
  Cube = Poly3DCollection(faces, facecolor=colors,
                          alpha=0.7)
  ax.set_xlabel('X')
  ax.set_ylabel('Y')
  ax.set_zlabel('Z')
  ax.set_xlim(0,1)
  ax.set_ylim(0,1)
  plt.show()
```

Result-

3D polygonal objects were succesfully created and

Output :



MULTIMEDIA APPLICATION INTEGRATES IMAGES, SOUND AND VEDIO IN UI

Aim- To develop a simple multimedia application that integrates images, vedio in UI

PROGRAM-

```
import tkinter as tk
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading
def image():
    img = image.open("sample_image.jpg").
        resize((300, 300))
    photo = ImageTk.PhotoImage(img)
    panel.config(image = photo)
    panel.image = photo
def audio():
    playsound("Sample_audio.mp3")
def vedio():
    cap = cv2.vedioCapture("Sample_vedio.
                           mp4")
    while cap.isopened():
        ret, frame = cap.read()
        cv2.imshow("vedio", frame)
        if (cv2.waitkey(25) & 0xFF == ord):
            break
cap.release()
cv2.destroyAllwindows()
root.title("Multimedia APP")
tk.Button(root, text = "Show Image", text =
    "play Audio", text = "play vedio")
root.mainloop()
```

RESULT- A multimedia app was succesfull

Output:



Ex no-14

CAPTURE VEDIO/AUDIO FROM WEBCAM OR MICROPHONE AND DISPLAY ON MULTIMEDIA

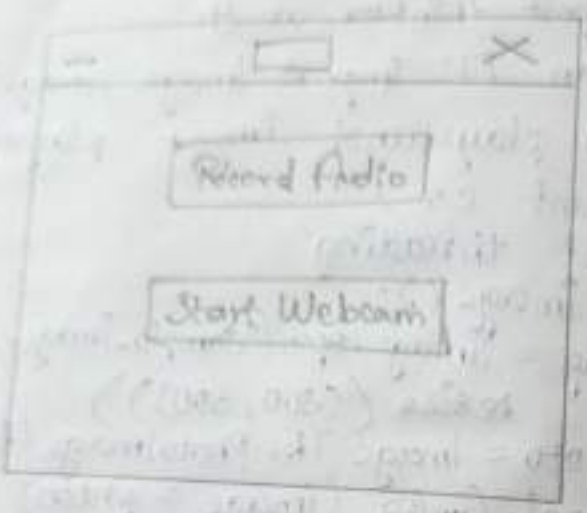
AIM- To develop a program that captures live vedio, audio from the webcam & microphones.

PROGRAM:

```
import tkinter as tk
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
def audio():
    fs = 44100
    print("Recording.....")
data = sd.rec(5 * fs, samplerate = fs,
    channels = 2)
sd.wait()
write("recorded_audio.wav", fs, data)
print("Audio saved!")
def video():
cap = cv2.videoCapture(0)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    cv2.imshow("Webcam", frame)
if cv2.waitKey(1) & 0XFF == ord('q'):
    break
Cap.release()
cv2.destroyAllwindows()
```

RESULT- Live vedio & audio were Successfully captured.

Output:



Ex no-
CAP
MICROP

AIM-
live
microp

PROGRA

data
Sd.
write(
pri
def
cap
while
ret
if
b
cv2
if cv2
break
Cap.rele
cv2.de

RESULT-
Success